

Churn Analysis and Customer Intelligence

```
In [27]: # Churn Analysis and Customer Intelligence
```

```
In [28]: # import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

1. Import database / data

```
In [ ]: # Data Import: db file to pandas, storing each table to a separate df

# Connect to SQLite database
conn = sqlite3.connect('customer_churn.db')

# sql query to Get all table names
sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into dataframe
    globals()[f"df_{table_name}"] = df                  # Create dynamic dataframe name
    print(f"Created dataframe: df_{table_name}")
```

```
# Close connection
conn.close()
```

Created dataframe: df_db_customer
 Created dataframe: df_db_subscription
 Created dataframe: df_db_support

```
In [ ]: # Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

Table Name: db_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription

Columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support

Columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

```
In [ ]: # PRAGMA is a special command in SQLite used to:
# inspect db information, control db settings, retrieve metadata about tables
```

2. Data cleaning

```
In [ ]: # df_db_customer.head() # customer table
df_db_customer.tail()
```

```
Out[ ]:
```

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	None	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	None	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	None	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	None	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	None	None

```
In [ ]: df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    object
1   name        21 non-null    object
2   country     18 non-null    object
3   state       21 non-null    object
4   gender      21 non-null    object
5   dob         21 non-null    object
6   interests   4 non-null     object
7   pincode     0 non-null     object
dtypes: object(8)
memory usage: 1.4+ KB
```

```
In [ ]: # a. rename col - name to customer_name
# b. drop columns - interest and pincode
# c. change data type - dob
# d. data standardization - gender
# e. fix missing values (using existing data) - country
```

a. Rename Col

```
In [ ]: # a. rename col - name to customer_name

df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
```

b. Drop columns

```
In [ ]: # b. drop columns - interest and pincode

# df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
# df_db_customer.drop(df_db_customer.columns[6:], axis=1)

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True) # using col name
```

```
In [ ]: df_db_customer.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    object
1   name         21 non-null    object
2   country      18 non-null    object
3   state        21 non-null    object
4   gender       21 non-null    object
5   dob          21 non-null    object
dtypes: object(6)
memory usage: 1.1+ KB
```

c. Change data type

```
In [ ]: # c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

d. Data standardization

```
In [ ]: # d. data standardization - gender

# df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Women' : 'Female'})
```

```
In [ ]: df_db_customer['gender'].unique()
```

```
Out[ ]: array(['Male', 'Female'], dtype=object)
```

e. Fix missing values

```
In [ ]: # e. fix missing values - country

# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]
```

```
Out[ ]:
```

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	None	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	None	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01

```
In [ ]: # country and state - unique value pair

# Creating state → country map from non-null rows
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()

# Fill the missing country using State
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping))
```

```
In [ ]: df_db_customer[df_db_customer['country'].isna()] # no null value in country col
```

```
Out[ ]:
```

	customerid	customer_name	country	state	gender	dob
--	------------	---------------	---------	-------	--------	-----

```
In [ ]:
```

```
In [ ]: df_db_subscription.head() # subscription table
```

Out []:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	n
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	None	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	None	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	

```
In [ ]: df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    object
1   subscription_start_date 21 non-null    object
2   subscription_type      21 non-null    object
3   renewal_date           21 non-null    object
4   plan_type              21 non-null    object
5   contract_type          21 non-null    object
6   cancellation_date       6 non-null     object
7   cancellation_reason     6 non-null     object
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

```
In [ ]: # change data type to date - subscription_start_date , renewal_date, cancellation_date
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    object
1   subscription_start_date 21 non-null    datetime64[ns]
2   subscription_type      21 non-null    object
3   renewal_date          21 non-null    datetime64[ns]
4   plan_type             21 non-null    object
5   contract_type         21 non-null    object
6   cancellation_date      6 non-null     datetime64[ns]
7   cancellation_reason    6 non-null     object
8   monthly_charges       21 non-null    float64
9   cltv                  21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB
```

```
In [ ]:
```

```
In [ ]: df_db_support.head() # support table
```

Out[]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	None
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	None

```
In [ ]: df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     object
1   complaint_date  9 non-null     object
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     object
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

```
In [ ]: # drop columns
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
In [ ]: # change data type to date - complaint_date

df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     object
1   complaint_date  9 non-null     datetime64[ns]
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 420.0+ bytes
```

```
In [ ]:
```


3. Feature Engineering & Data Analysis

Create a new col

```
In [ ]: # create a new col using existing col - churn flag

# Customer is churned if cancellation_date is not null
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date'].notna(), 1, 0)
df_db_subscription.head()
```

Out []:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	n
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	None	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	

```
In [ ]:
```

Merge Dataframes - JOINS

```
In [ ]: # first fix support table duplicates then merge
# Note: while merging df's always check the shape before and after
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left') )
```

```
In [ ]: df_db_subscription.shape
```

```
Out[ ]: (21, 12)
```

```
In [ ]: df.shape
```

```
Out[ ]: (23, 20)
```

```
In [ ]: print('df_db_subscription unique value:', df_db_subscription['customerid'].nunique())  
print('df_db_customer unique value:', df_db_customer['customerid'].nunique())  
print('df_db_support unique value:', df_db_support['customerid'].nunique())  
print('df_db_support all value:', df_db_support['customerid'].size)
```

```
df_db_subscription unique value: 21
```

```
df_db_customer unique value: 21
```

```
df_db_support unique value: 7
```

```
df_db_support all value: 9
```

```
In [ ]: df_db_support
```

```
Out[ ]:
```

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

```
In [ ]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'].transform('count')
```

```
In [ ]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid', keep= 'last')
```

```
In [ ]: df_db_support['customerid'].size
```

```
Out[ ]: 7
```

```
In [ ]: # merge df
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left') )
```

```
In [ ]: df.shape
```

```
Out[ ]: (21, 21)
```

Export dataframe

```
In [ ]: # export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)
```

```
In [ ]:
```

Data Analysis

```
In [ ]: # 1. Churn Rate
```

```
In [ ]: churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

Churn Rate = 28.57 %

```
In [ ]: # 2. Retention Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

Retention Rate = 71.43 %

```
In [ ]: # 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn_rate_pct'))
print(churn_by_plan)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [ ]: # 4 a. Churn by state + sum(revenue) & count of users -- home work **
# 4 b. Churn by subscription type + sum(revenue) & count of users -- home work **
```

```
In [ ]: # 5. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

ARPU = 18.85

```
In [ ]: # homework - calculate customer age
```

```
In [ ]: # 6. Avg Customer Tenure
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

Avg Tenure (Days) = 1452 0

```
In [ ]: # 7. Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
```

```
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

Revenue at Risk (Rs 'K') = 73.94

```
In [ ]: # 8. Escalation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

Escalation Rate = 19.05 %

```
In [ ]: # 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

Avg Compliants Per User = 0.43

```
In [ ]: # 10. Correlation Escalation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to int type
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation,2))
```

Correlation between escalation vs churn is = 0.77

```
In [ ]: # 11. Create a column using existing col - Churn risk
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unkown')
```

```
In [ ]:
```

4. Visualization using Matplotlib

```
In [ ]: # best practice to create a copy then work on it
df_visual = df.copy()
```

```
In [ ]: df_visual.columns
```

```
Out[ ]: Index(['customerid', 'subscription_start_date', 'subscription_type',
              'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
              'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
              'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
              'complaint_date', 'escalations', 'csat_score', 'complaint_count',
              'tenure_days', 'churn_risk'],
              dtype='object')
```

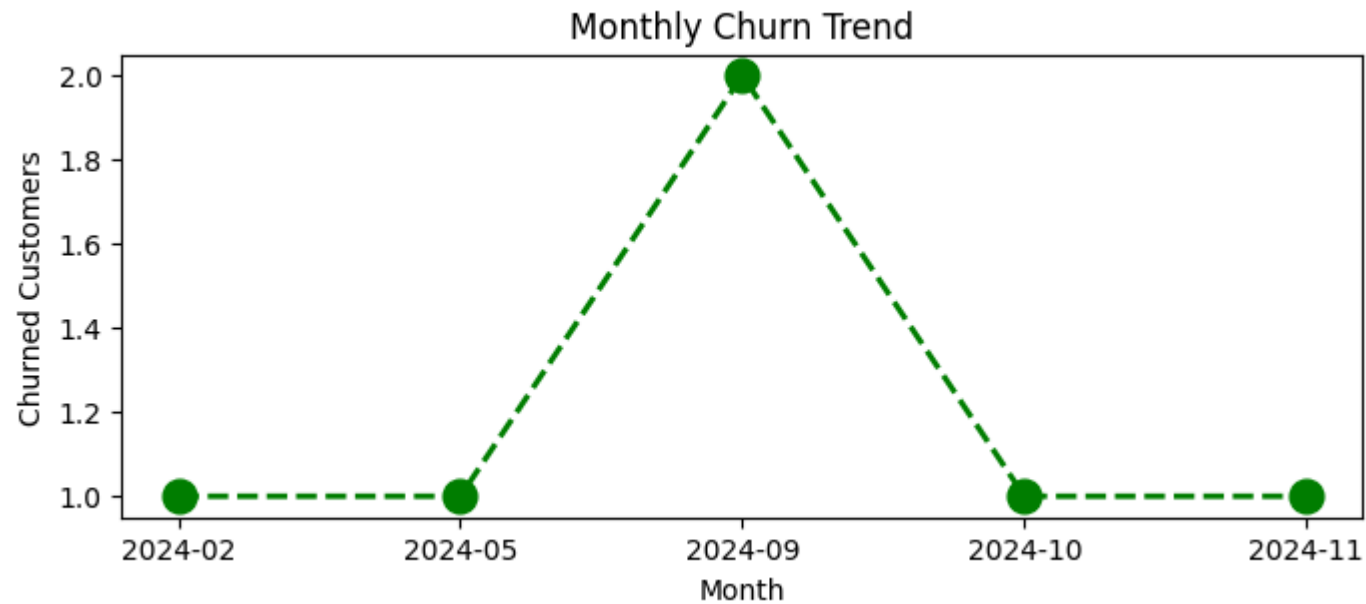
```
In [ ]: # 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker='o', linestyle='dashed', linewidth=2, mark

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```

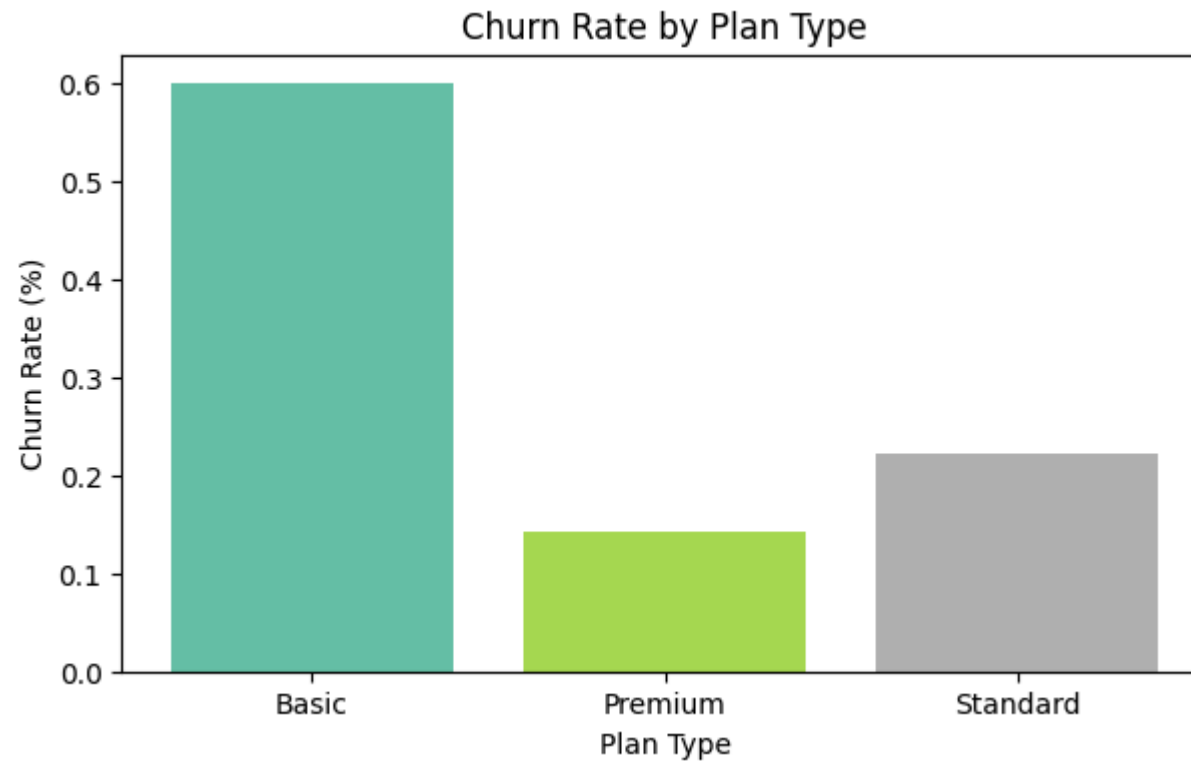


```
In [ ]: # 4.2 Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```

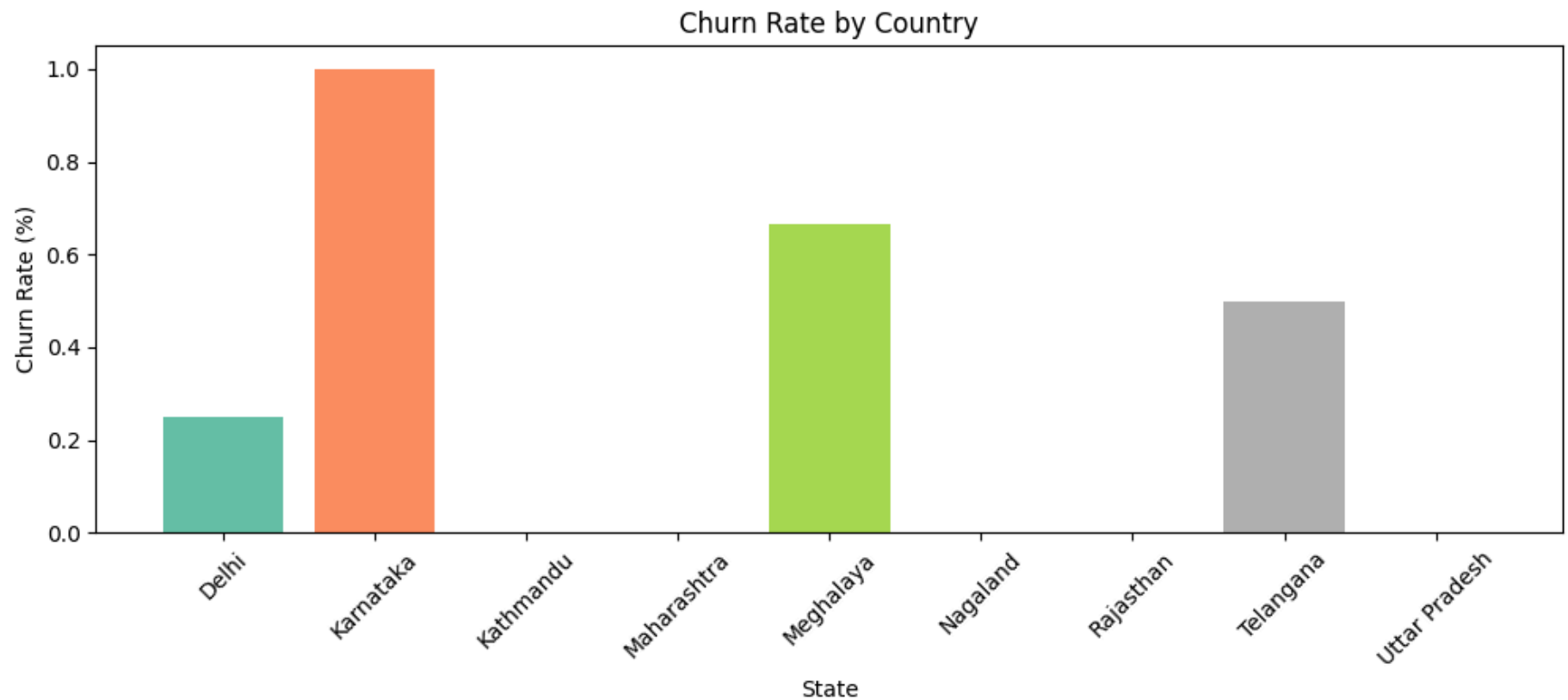


```
In [ ]: # 4.3 Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```

In []:

Visulaization using Seaborn

In []: *# Heatmap (Correlation Matrix)*

In []: *# encoding - convert str to numeric so that we can find corr between features*
df_visual.columns

```
Out[ ]: Index(['customerid', 'subscription_start_date', 'subscription_type',
              'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
              'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
              'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
              'complaint_date', 'escalations', 'csat_score', 'complaint_count',
              'tenure_days', 'churn_risk', 'cancellation_month'],
              dtype='object')
```

```
In [ ]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']].head()
```

```
Out[ ]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

```
In [ ]: # remove warnings
import warnings
warnings.filterwarnings("ignore")
```

```
In [ ]: # incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]

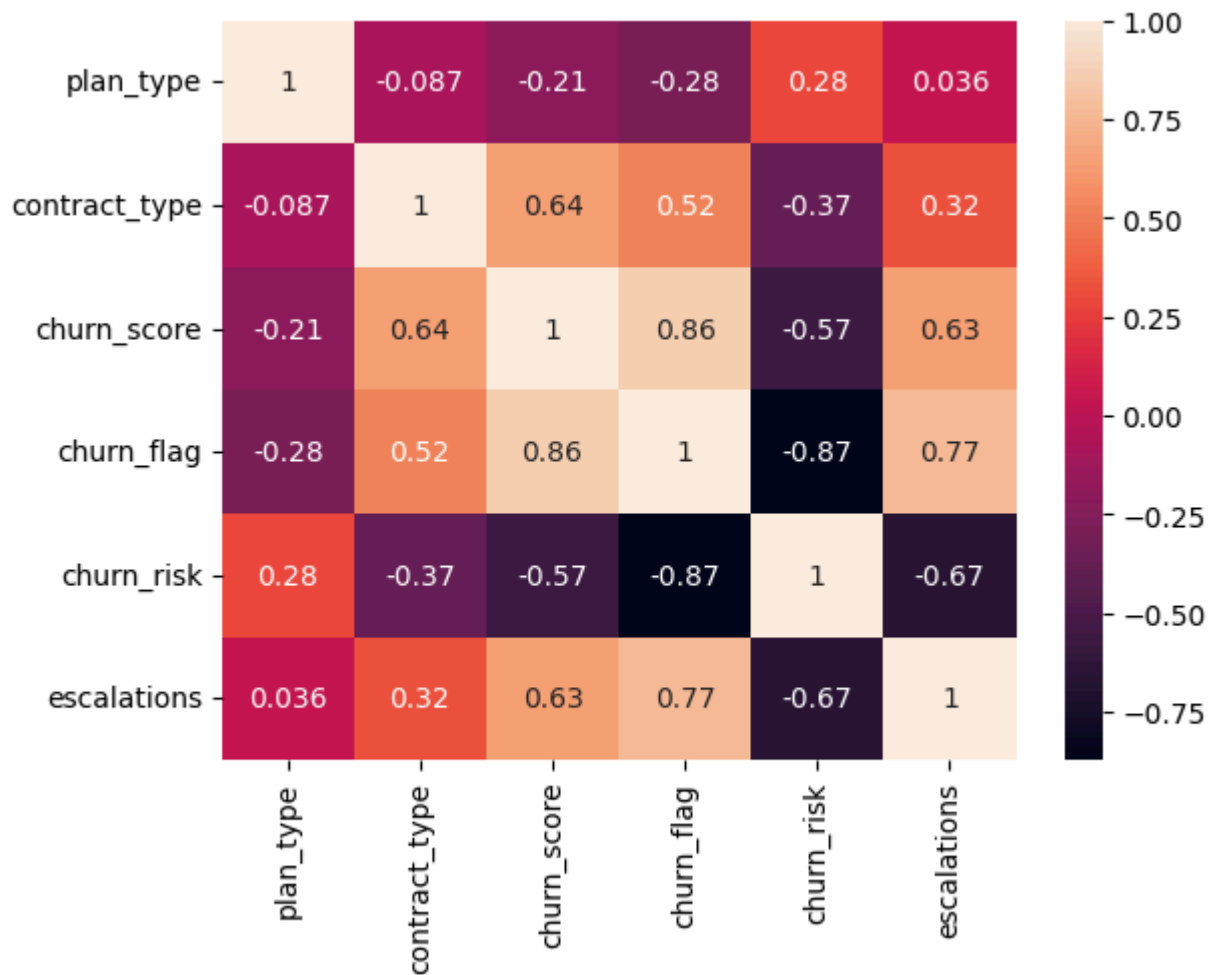
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
In [ ]: # Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

```
Out[ ]: <Axes: >
```



```
In [ ]: print('plan_type :', df_visual['plan_type'].unique())
        print('contract_type :', df_visual['contract_type'].unique())
        print('churn_risk :', df_visual['churn_risk'].unique())
```

```
plan_type : ['Standard' 'Premium' 'Basic']
contract_type : ['Annual' 'Monthly']
churn_risk : ['low' 'high' 'med']
```

```
In [ ]: # Correct method of encoding - based on priority
        df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]
```

```

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'med', 'high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order, ordered=True).codes

```

```
In [ ]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']].head()
```

```
Out [ ]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

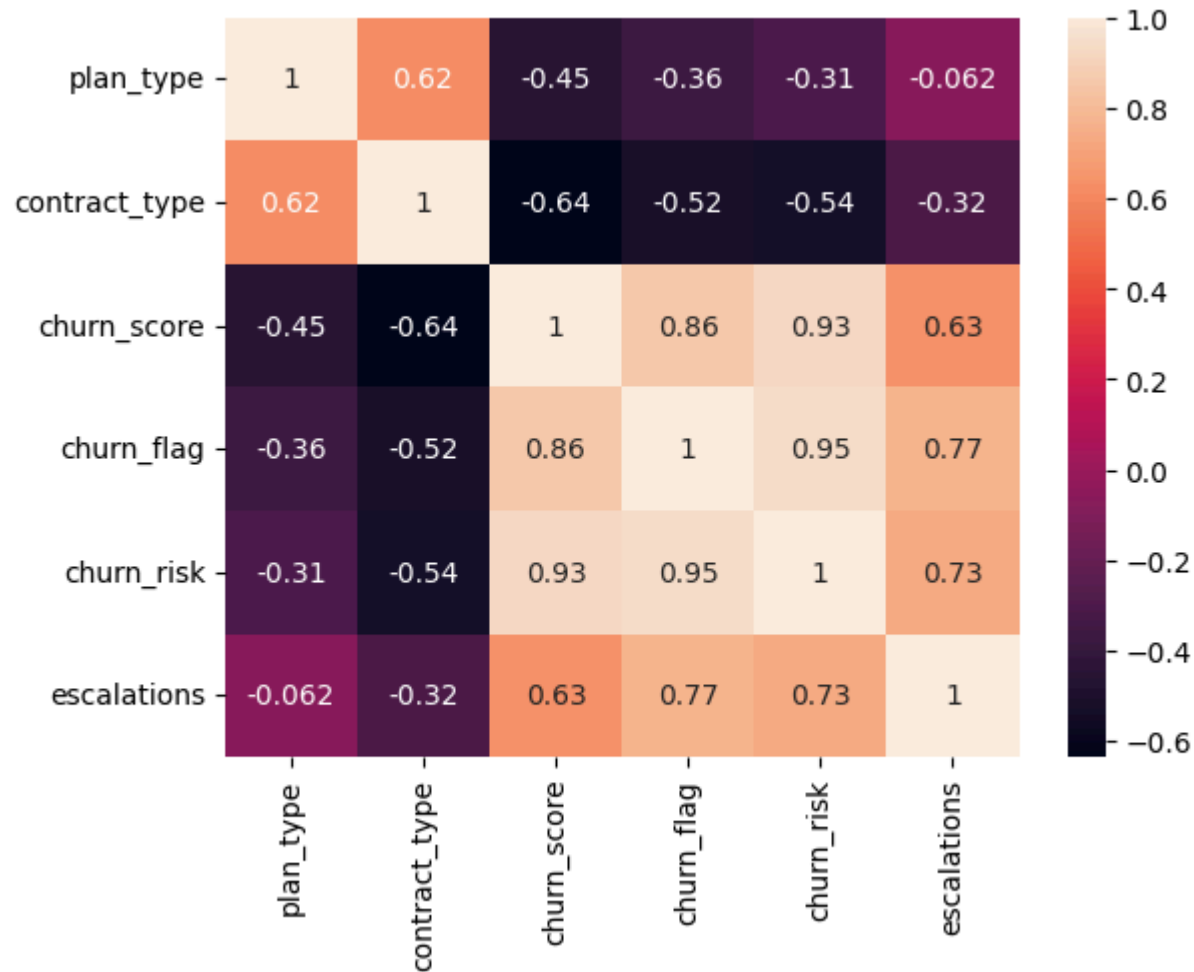
```
In [ ]: df_encoded.head()
```

```
Out [ ]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	1

```
In [ ]: # Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[]: <Axes: >



```
In [ ]: # Heatmap Using Matplotlib - difficult to plot it

corr_matrix = df_encoded.corr() # Correlation matrix

fig, ax = plt.subplots(figsize=(10, 6)) # Create figure

cax = ax.imshow(corr_matrix, cmap='coolwarm') # Heatmap
```

```
fig.colorbar(cax) # Add colorbar

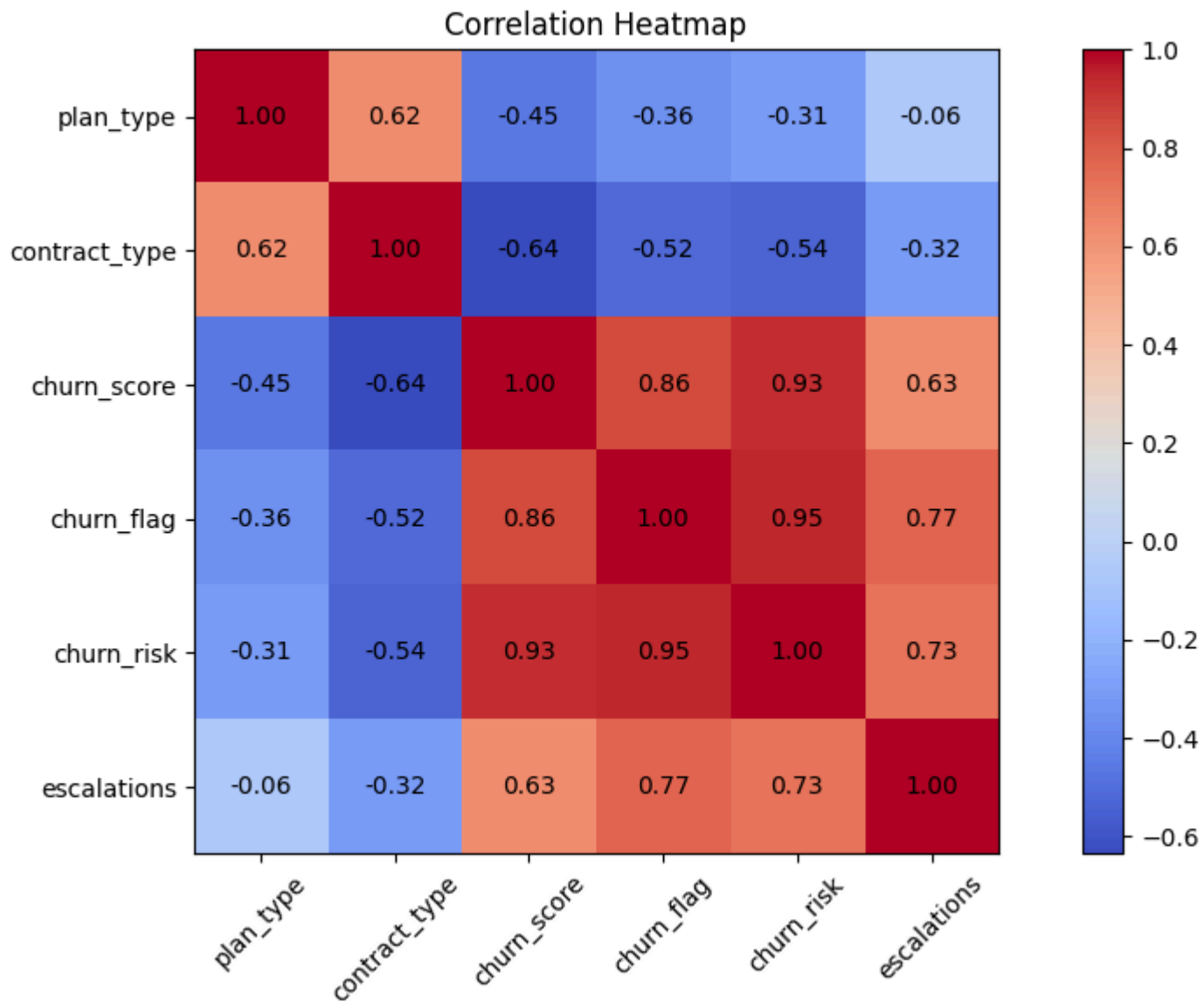
# Axis Labels
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))

ax.set_xticklabels(corr_matrix.columns, rotation=45)
ax.set_yticklabels(corr_matrix.columns)

# Annotate values inside cells
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        ax.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha='center',
            va='center'
        )

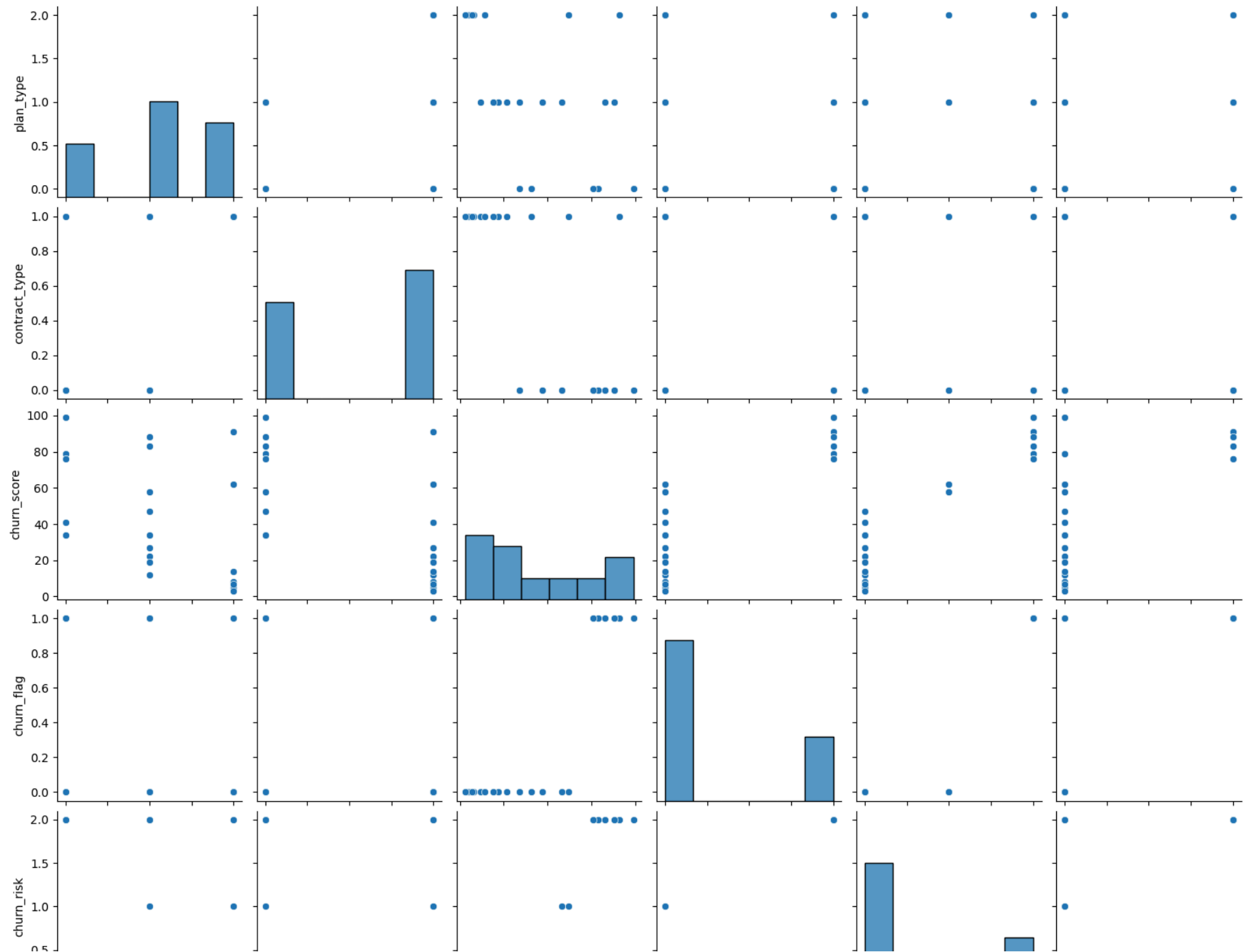
plt.title('Correlation Heatmap') # Title

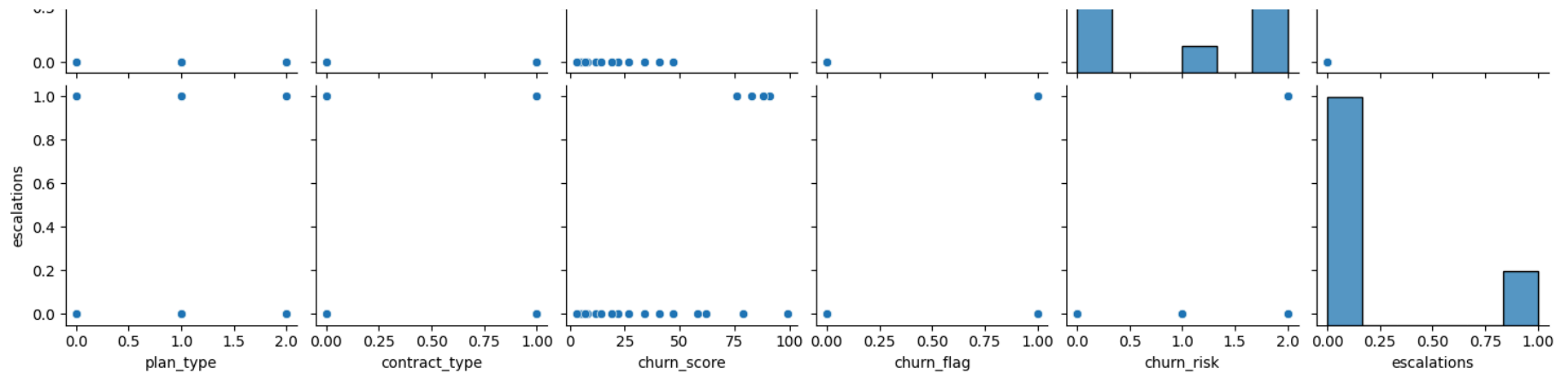
plt.tight_layout()
plt.show()
```



```
In [ ]: # pairplot - Plot pairwise relationships in a dataset
sns.pairplot(df_encoded)
```

```
Out[ ]: <seaborn.axisgrid.PairGrid at 0x1d38f71ef90>
```





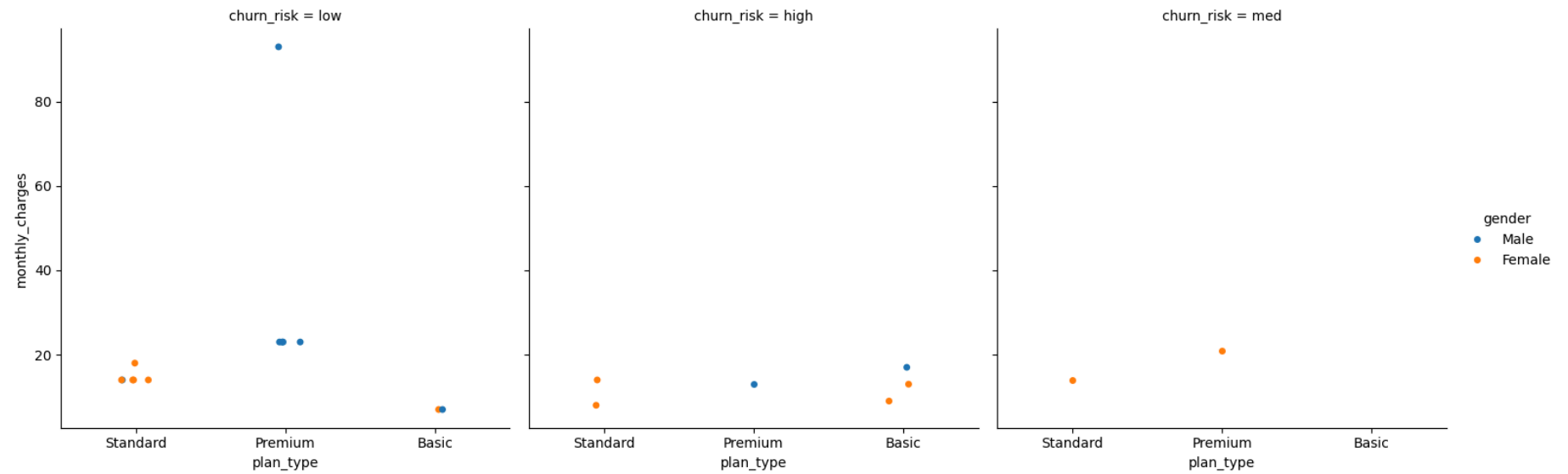
```
In [ ]: df_visual.columns
```

```
Out[ ]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count',
               'tenure_days', 'churn_risk', 'cancellation_month'],
              dtype='object')
```

```
In [ ]: # catplt/Facegrid plot - multi-dim comparison
```

```
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

```
Out[ ]: <seaborn.axisgrid.FacetGrid at 0x1d38f746c90>
```



In []:

Pivot table

In []:

```
# Pivot Table
pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
)
```

Out[]:

churn_flag	
plan_type	
Basic	0.600000
Premium	0.142857
Standard	0.222222

In []: *# pivot table using multiple cols and agg type*

```
pd.pivot_table(  
    df_visual,  
    index='plan_type',  
    values=['monthly_charges', 'customerid', 'churn_flag'],  
    aggfunc = {  
        'monthly_charges' : 'sum',  
        'customerid' : 'nunique',  
        'churn_flag' : 'mean'  
    }  
)
```

Out[]:

churn_flag customerid monthly_charges			
plan_type			
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

In []:

Project Completed :)

```
In [ ]: # Project Completed Here. ALL the best! :)
```

```
In [ ]:
```

Working with SQL in Python

```
In [ ]: # create db in sql
conn = sqlite3.connect('test_database.sqlite')

#table details
conn.execute("CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)")

# commit and save
conn.commit()

print("Cretaed Table successfully!")
```

Cretaed Table successfully!

```
In [ ]: import sqlite3

# Connect to database
conn = sqlite3.connect("customer_churn.db")

# Create cursor
cursor = conn.cursor()

# Create users table if it doesn't exist
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    name TEXT,
    country TEXT,
    salary INTEGER
)
""")

# Insert records
cursor.execute("""
```

```
INSERT INTO users (name, country, salary)
VALUES
    ('Anish', 'India', 5000),
    ('Manish', 'Germany', 2500),
    ('Mukesh', 'India', 3500)
""")

# Save changes
conn.commit()

print("Data inserted successfully!")

# Display inserted data
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())

# Close connection
conn.close()
```

Data inserted successfully!

```
[('Anish', 'India', 5000), ('Manish', 'Germany', 2500), ('Mukesh', 'India', 3500)]
```

```
In [31]: # Connect to the SQLite database
conn = sqlite3.connect("test_database.sqlite")

# SQL query
query = "SELECT * FROM users;"

# Read data into a DataFrame
df_results = pd.read_sql_query(query, conn)

# Display the first 5 rows
print(df_results.head())

# Close the connection
conn.close()
```

Empty DataFrame

Columns: [first_name, country, budget]

Index: []

```
In [30]: # Connect to the database
conn = sqlite3.connect("customer_churn.db") # or "test_database.sqlite"

# SQL query
query = """
SELECT
    country,
    SUM(salary) AS total_salary
FROM users
GROUP BY country;
"""

# Read results into DataFrame
df_agg = pd.read_sql_query(query, conn)

# Display results
print(df_agg)

# Close connection
conn.close()
```

	country	total_salary
0	Germany	2500
1	India	8500

```
In [ ]: # always close the conn with db once the task is over
conn.close()
```